

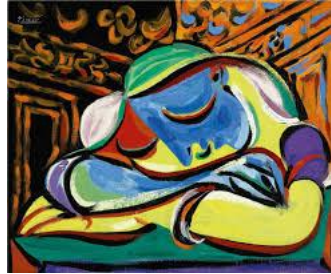
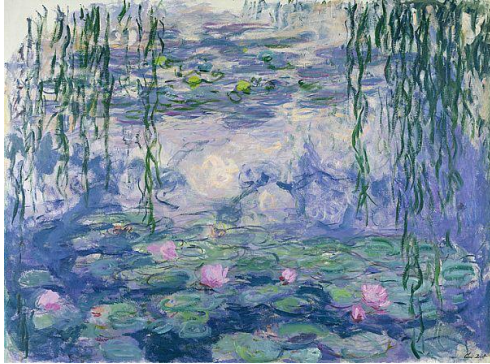


UNIVERSITY OF
TORONTO

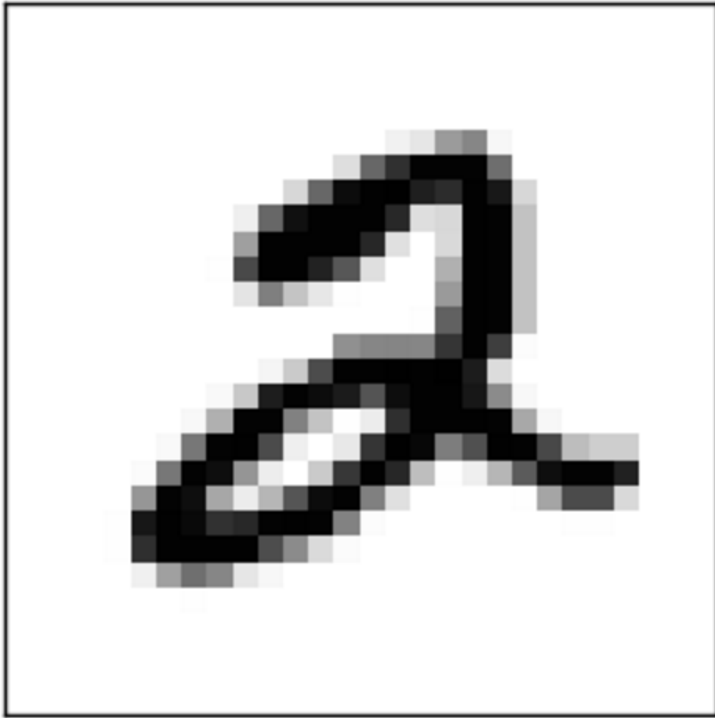
Engineering

Convolutional Networks

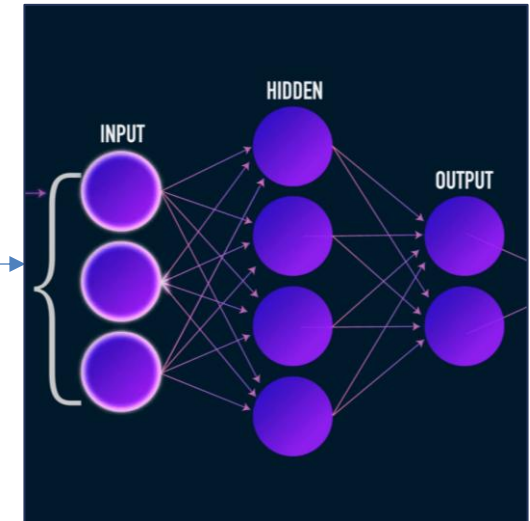
Deep Learning



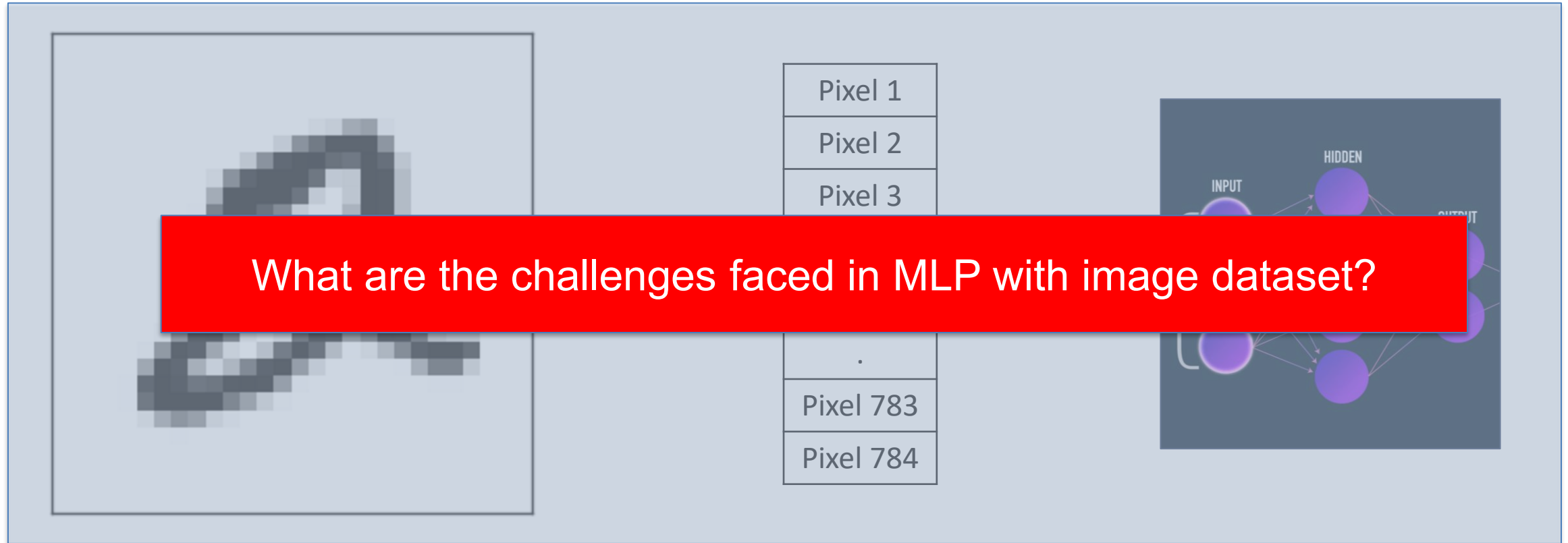
MLP with Image data



Pixel 1
Pixel 2
Pixel 3
.
.
.
Pixel 783
Pixel 784



MLP with Image data



Challenges

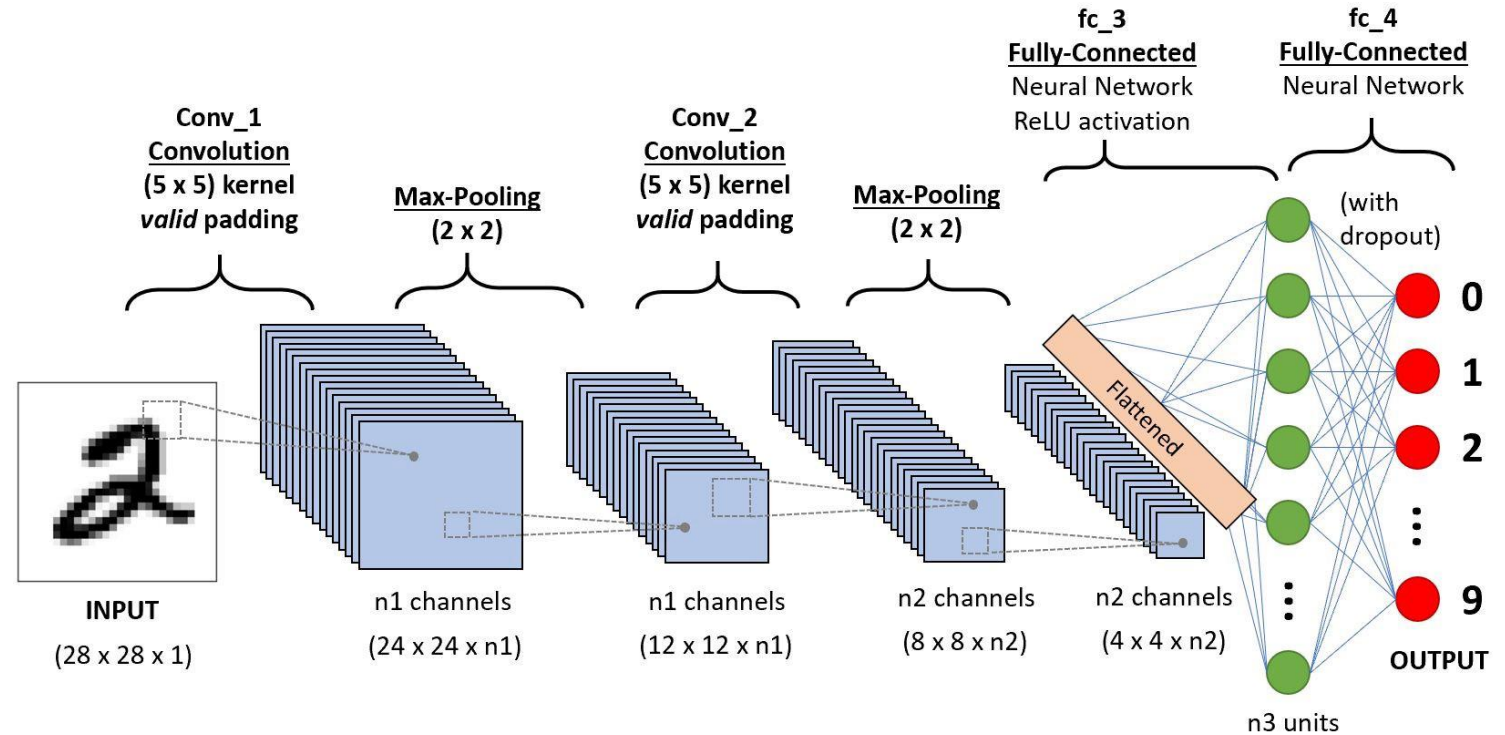
- ❖ Computation time will be higher
- ❖ Large number of weights
- ❖ Each pixel has its own associated weight. Small shifts in image can result in large change in prediction.
- ❖ Does not exploit geometric relationships in image.

General Architecture of CNN

Local connection

Weight Sharing

Learned feature
extraction



Steps in 2D Convolution

- ❖ Place the kernel on a patch of the image.
- ❖ Multiply each kernel value by the pixel underneath, then sum all the products → one output value.
- ❖ Slide the kernel across the entire image, repeating step 2 → the resulting grid is called a feature map.

Strictly speaking, this is "cross-correlation" — classical convolution flips the kernel first. Deep learning frameworks (e.g., PyTorch's Conv2d) skip the flip but still call it convolution. Since the kernel values are **learned**, the flip makes no practical difference.

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

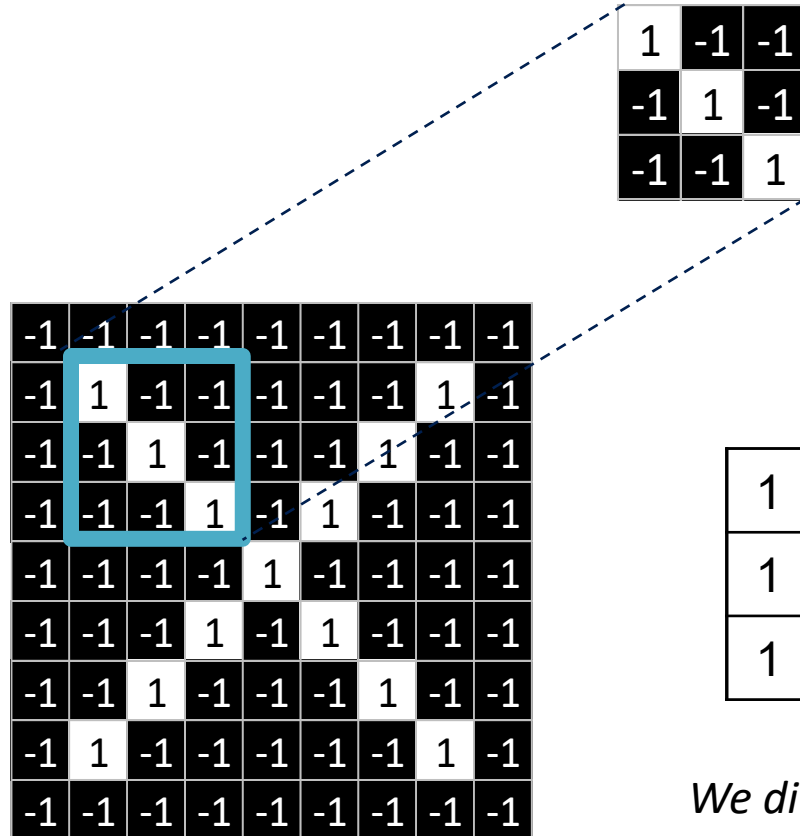
Image

1	-1	1
-1	1	-1
1	-1	1

Kernel

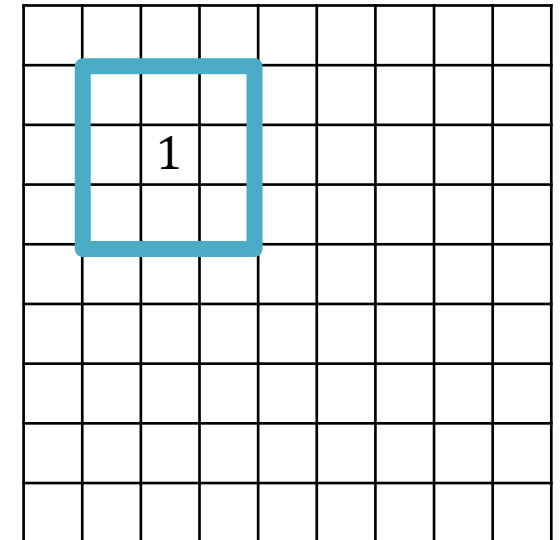
Steps in CNN

- Filtering



1	1	1
1	1	1
1	1	1

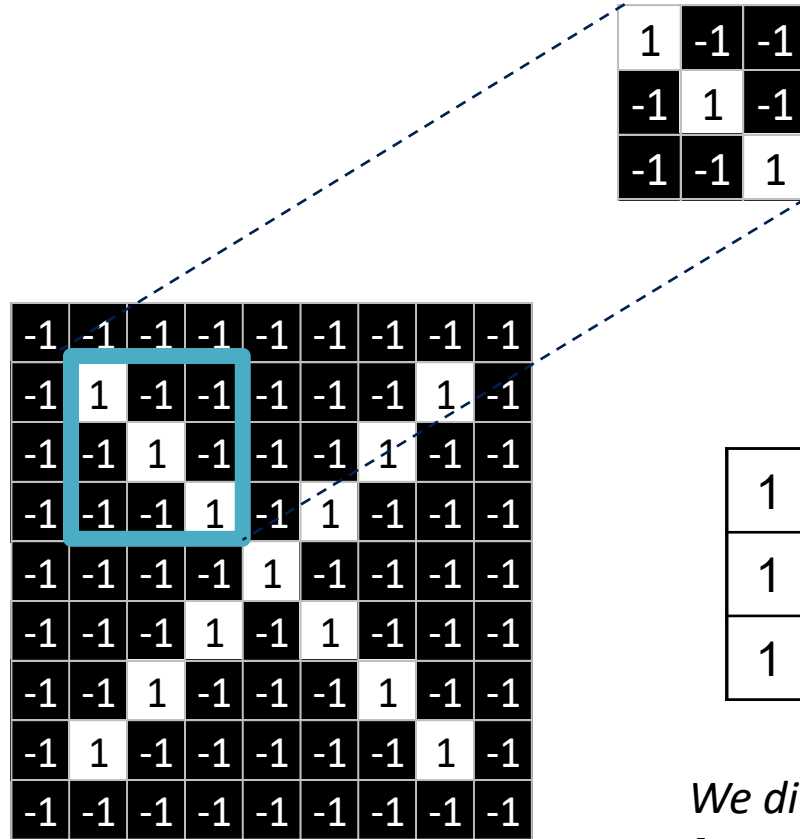
$$\frac{1 + 1 + \dots + 1}{9} = 1$$



We divide by 9 here only to keep values in $[-1, 1]$ for this toy example. Real conv layers just sum (plus a learned bias), i.e., no normalization.

Steps in CNN

- Filtering



1	1	1
1	1	1
1	1	1

$$\frac{1 + 1 + \dots + 1}{9} = 1$$

We divide by 9 here only to keep values in $[-1, 1]$ for this toy example. Real conv layers just sum (plus a learned bias), i.e., no normalization.

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

Convolution

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1



1	-1	-1
-1	1	-1
-1	-1	1

=

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

Convolution on Multiple Filters

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1



1	-1	1
-1	1	-1
1	-1	1

=

0.33	-0.55	0.11	-0.11	0.11	-0.55	0.33
-0.55	0.55	-0.55	0.33	-0.55	0.55	-0.55
0.11	-0.55	0.55	-0.77	0.55	-0.55	0.11
-0.11	0.33	-0.77	1.00	-0.77	0.33	-0.11
0.11	-0.55	0.55	-0.77	0.55	-0.55	0.11
-0.55	0.55	-0.55	0.33	-0.55	0.55	-0.55
0.33	-0.55	0.11	-0.11	0.11	-0.55	0.33

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

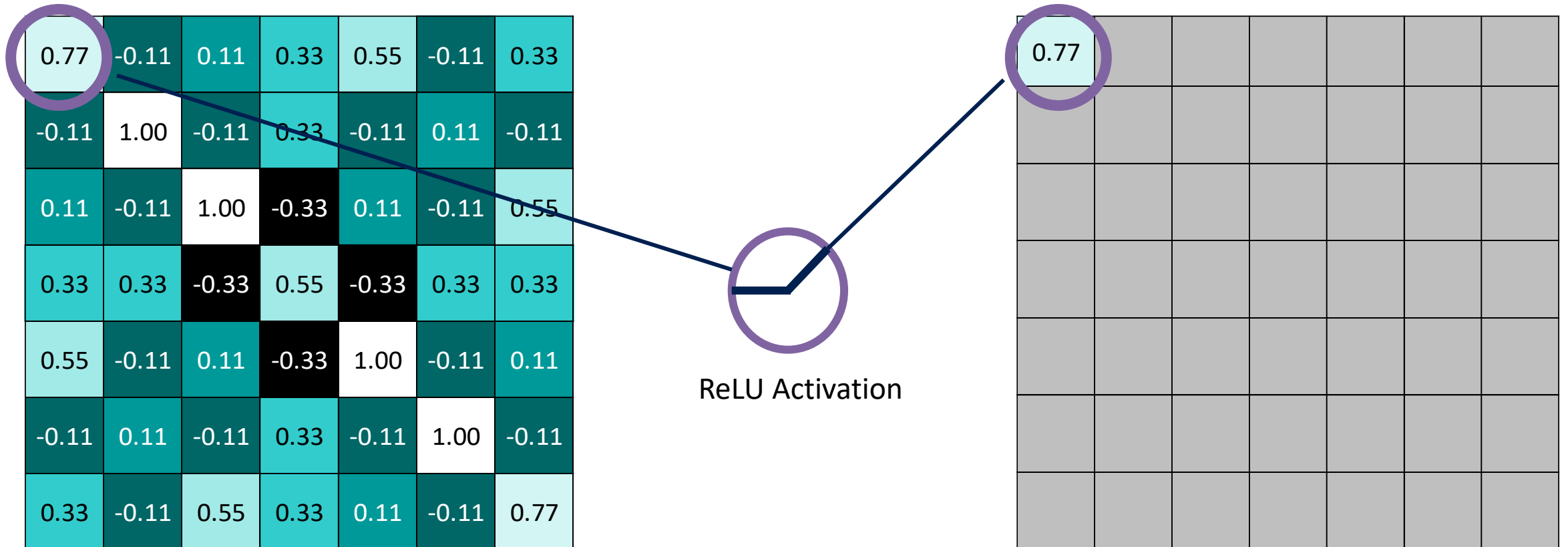


-1	-1	1
-1	1	-1
1	-1	-1

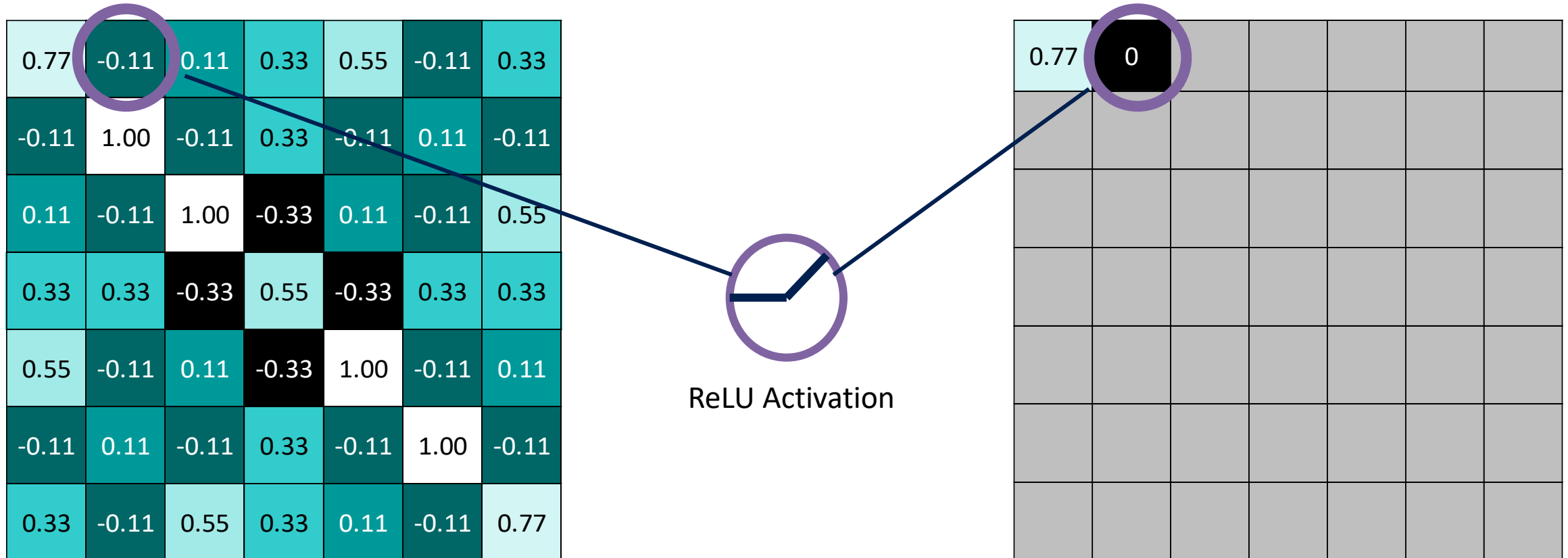
=

0.33	-0.11	0.55	0.33	0.11	-0.11	0.77
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.77	-0.11	0.11	0.33	0.55	-0.11	0.33

Activation

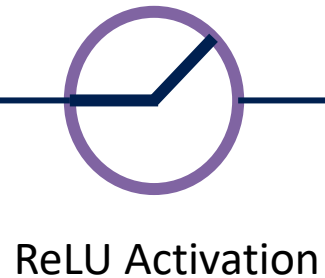


Activation



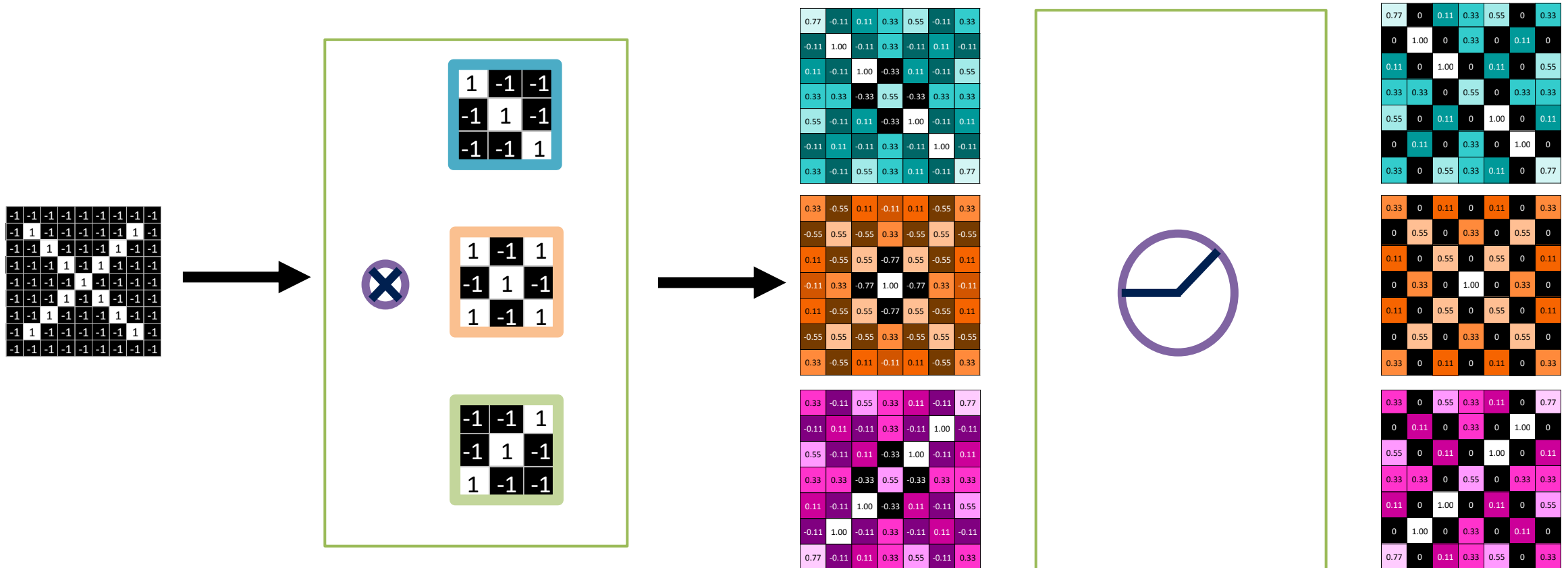
Activation

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77



0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77

Convolution and Activation



Pooling

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77

maximum
 $\max\{0.77, 0, 0, 1.00\} \rightarrow 1.00$

1.00			

Pooling

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.00	-0.11	0.33	-0.11	-0.11	-0.11
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

maximum
 $\max\{0.11, 0.33, -0.11, 0.33\} \rightarrow 0.33$

1.00	0.33		

Pooling

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

Maximum

1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

Pooling on All Channels

0.33	-0.55	0.11	-0.11	0.11	-0.55	0.33
-0.55	0.55	-0.55	0.33	-0.55	0.55	-0.55
0.11	-0.55	0.55	-0.77	0.55	-0.55	0.11
-0.11	0.33	-0.77	1.00	-0.77	0.33	-0.11
0.11	-0.55	0.55	-0.77	0.55	-0.55	0.11
-0.55	0.55	-0.55	0.33	-0.55	0.55	-0.55
0.33	-0.55	0.11	-0.11	0.11	-0.55	0.33

0.33	-0.11	0.55	0.33	0.11	-0.11	0.77
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.77	-0.11	0.11	0.33	0.55	-0.11	0.33

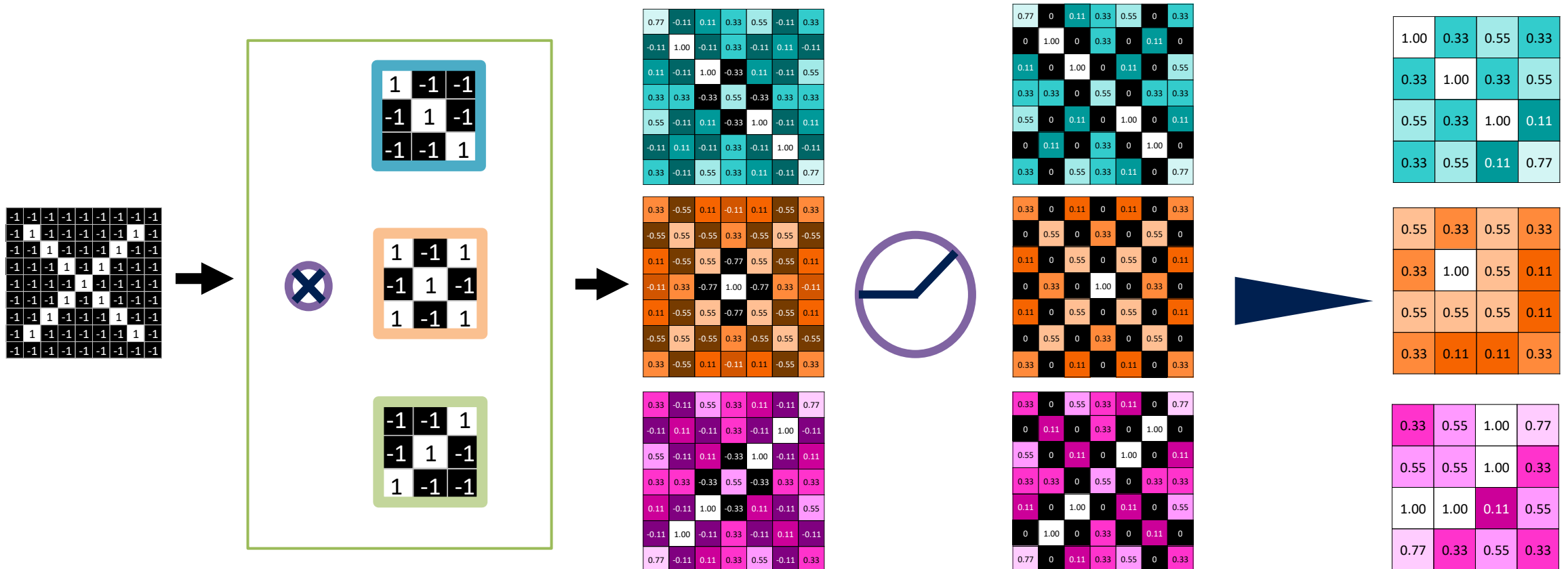


0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33

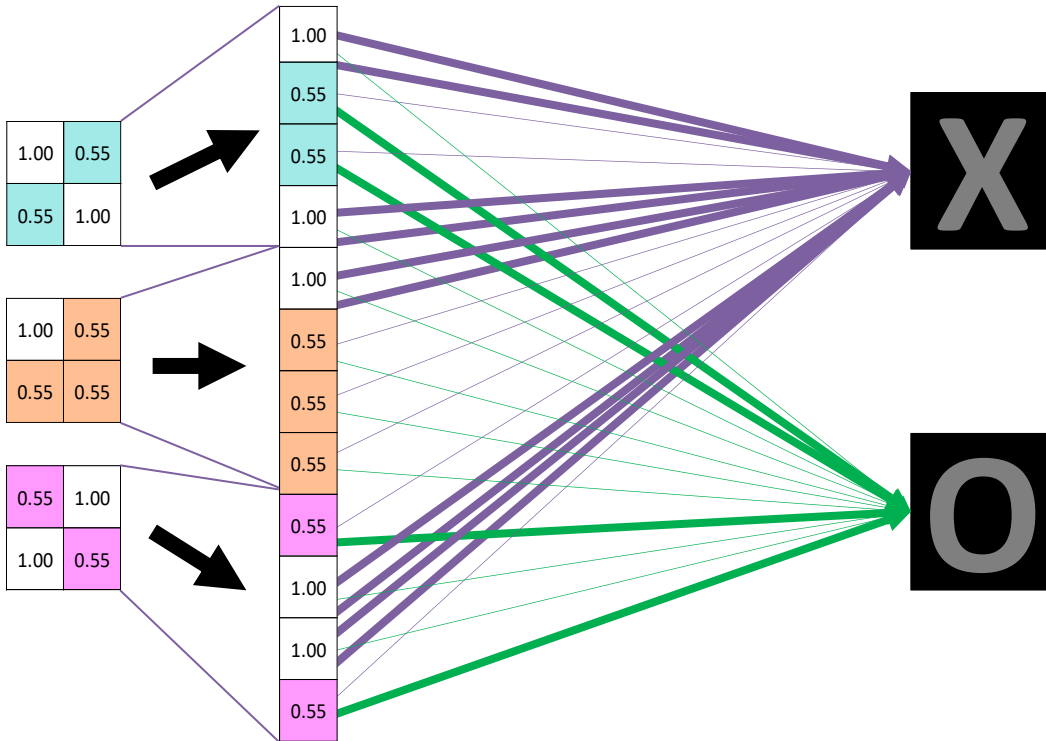


0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

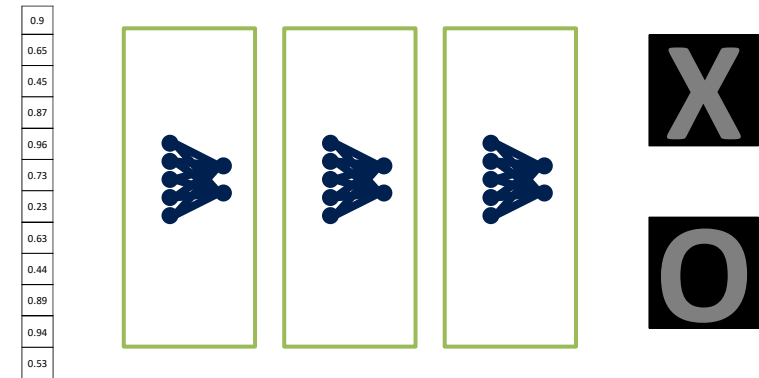
Convolution and Pooling over 3 Channels



Classification

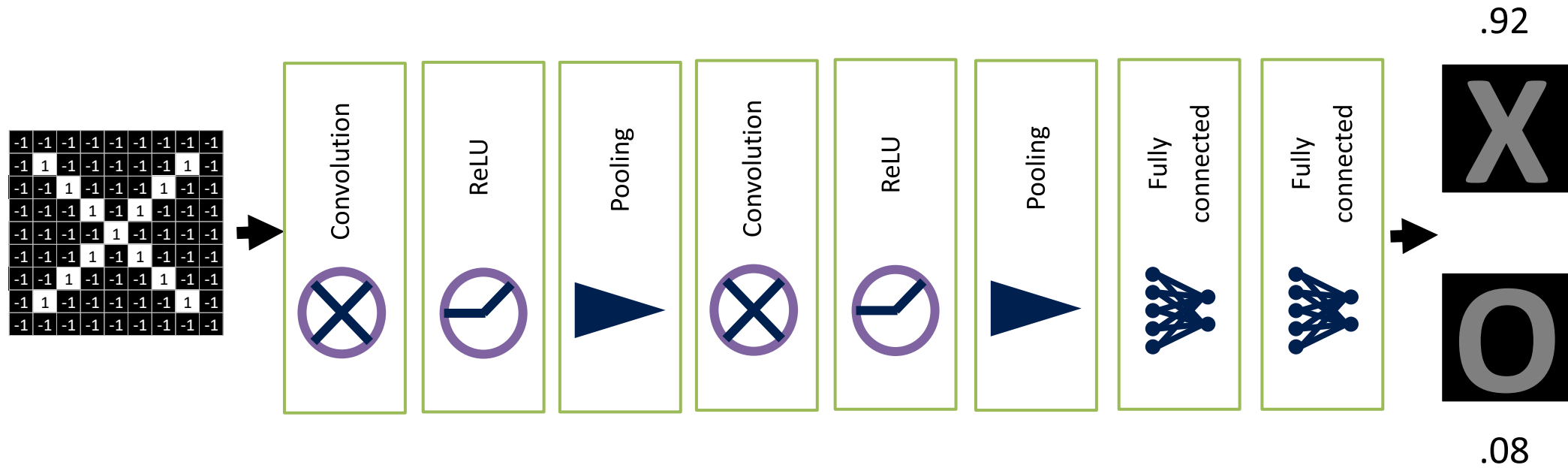


- Fully Connected Layer
 - Flattening
 - Binary classification
 - Cross entropy loss
 - Multiple layers before output



Steps in CNN

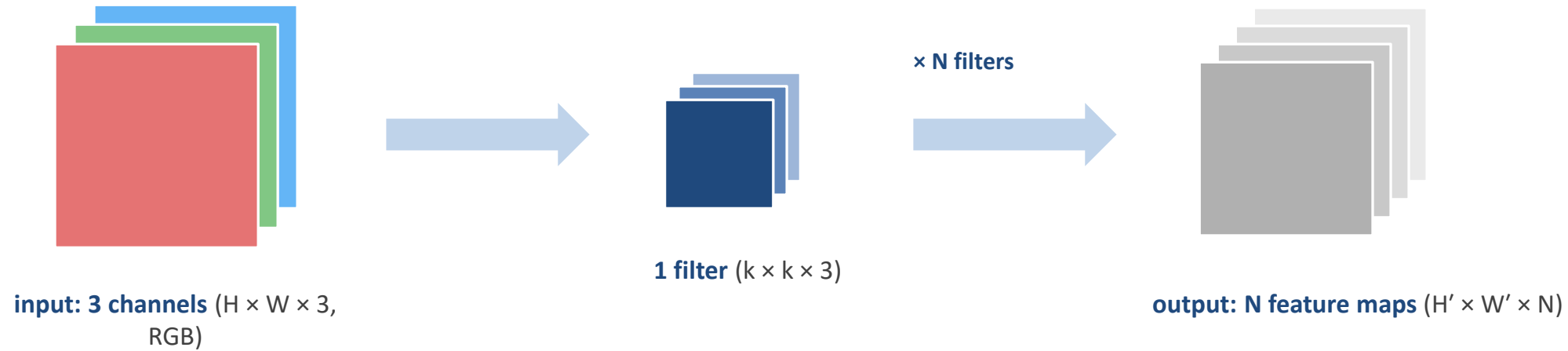
- Architecture of CNN



What if the image has 3 dimensions?

- ❖ The kernel becomes 3-dimensional (e.g., $3 \times 3 \times 3$)
- ❖ The output would still be a single value.

Channels, Filters, Feature Maps



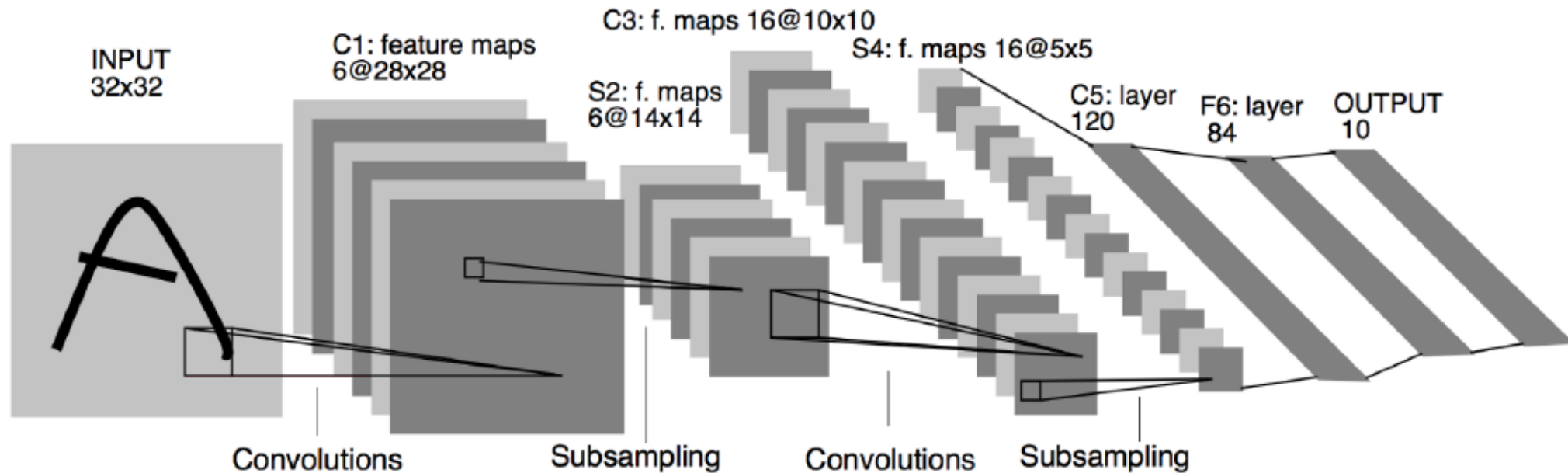
- ❖ **Channels** = the depth of a tensor. Input image: 3 channels (RGB) or 1 (grayscale).
- A **filter** always spans ALL input channels ($k \times k \times C_{in}$) and gives ONE number per position.
- 1 filter $\rightarrow$ 1 **feature map**. N filters $\rightarrow$ N output channels — feature maps are the channels of the next layer.
- In the earlier X/O example, the “3 channels” after convolution were 3 filters’ outputs (the input was 1-channel grayscale).

```
nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3) # 16 filters, each  $3 \times 3 \times 3$ 
```


CNN Architectures

- ❖ LeNet-5 — LeCun et al., 1998: the proof of concept (handwritten digits)
- ❖ AlexNet — 2012: deep learning's breakout — ImageNet winner, built at the University of Toronto
- ❖ VGGNet — 2014: depth through simplicity (uniform 3×3 stacks) — ImageNet runner-up
- ❖ GoogLeNet (Inception) — 2014: efficiency through smarter modules — ImageNet winner
- ❖ ResNet — 2015: the depth barrier falls (skip connections) — ImageNet winner, default backbone today

LeNet Architecture

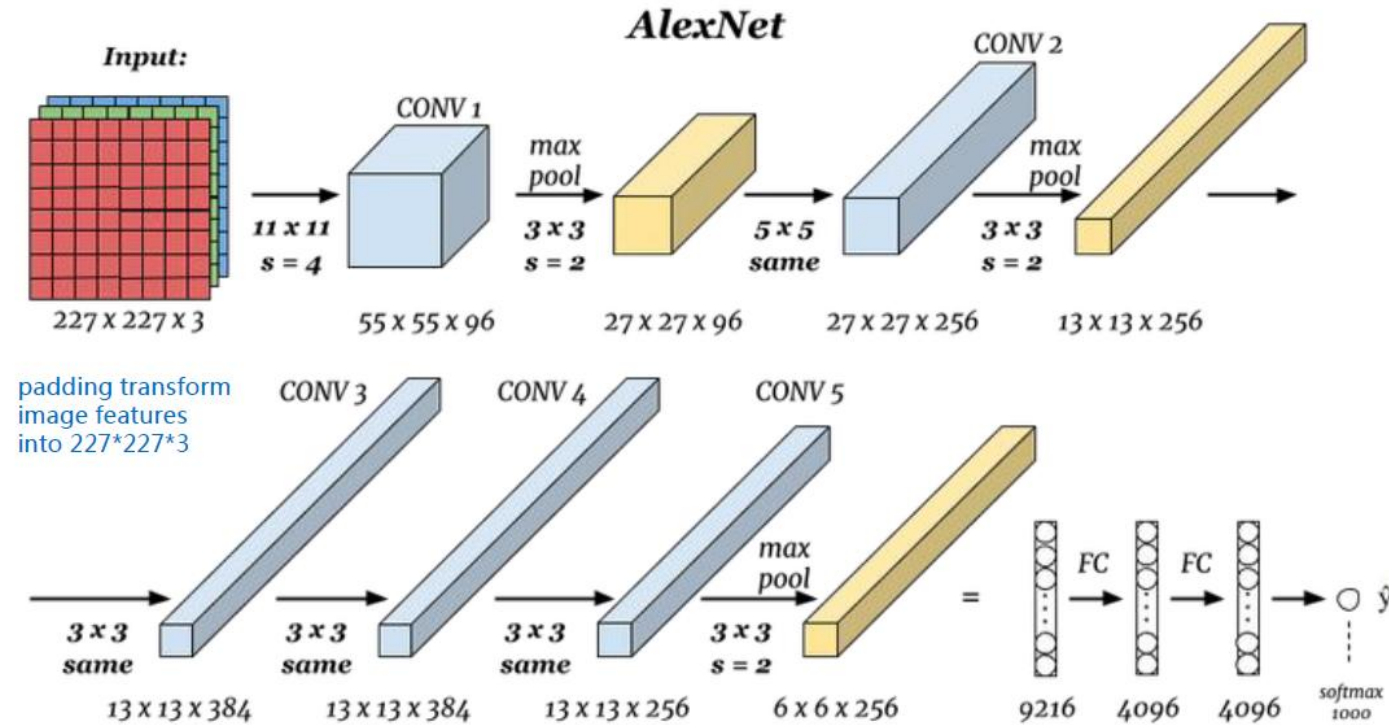


Why: prove a neural network can read images end-to-end — deployed by banks to read cheques

Breakthrough: the CNN recipe — convolution + pooling + fully-connected, all trained by backprop (~60K params)

Limitation: 1990s data & compute — works on 32x32 digits, far from real-world images

AlexNet Architecture



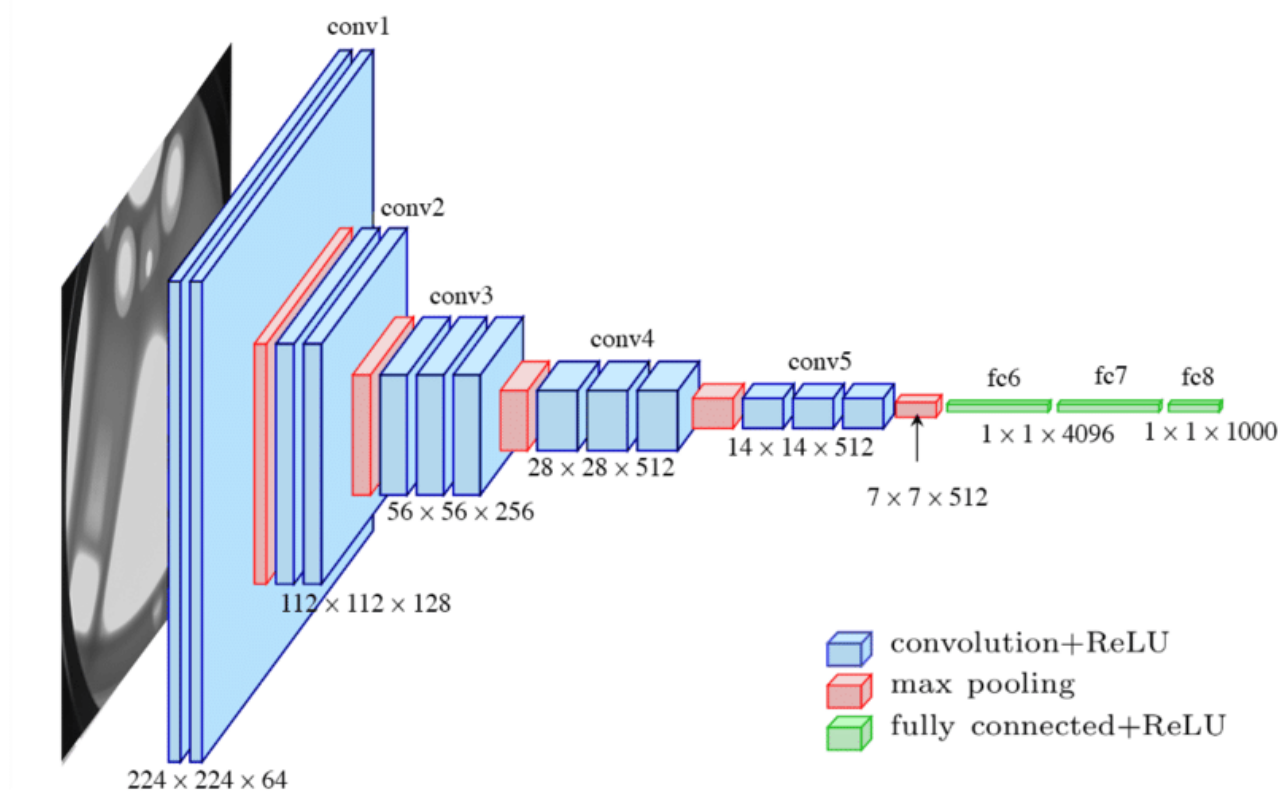
Built here at U of T — Krizhevsky, Sutskever & Hinton (2012)

Why: hand-crafted features had plateaued on ImageNet (1.2M images, 1,000 classes)

Breakthrough: first deep CNN at scale — ReLU + dropout + 2 GPUs → top-5 error 26% → 15%

Limitation: ad-hoc design (11x11 filters); the FC layers hold most of its 60M parameters

VGGNet Architecture

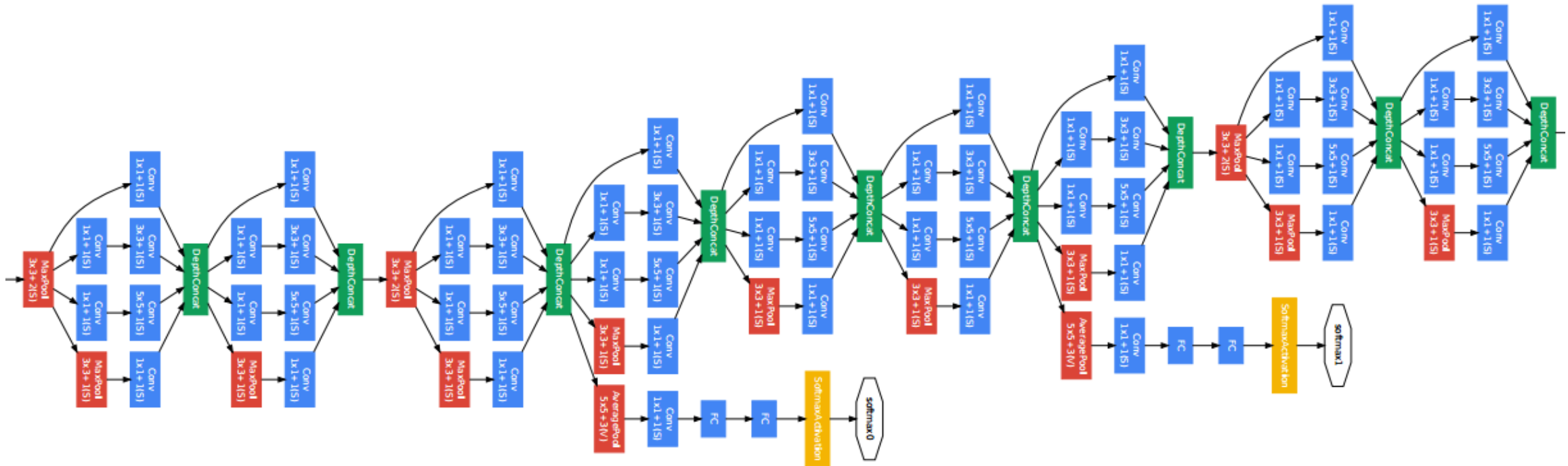


Why: after AlexNet — does simply going deeper keep helping?

Breakthrough: uniform 3x3 convs only, 16–19 layers; two 3x3 $\approx$ one 5x5 with fewer params + more nonlinearity

Limitation: brute-force depth — 138M parameters, slow and memory-hungry

Inception Architecture

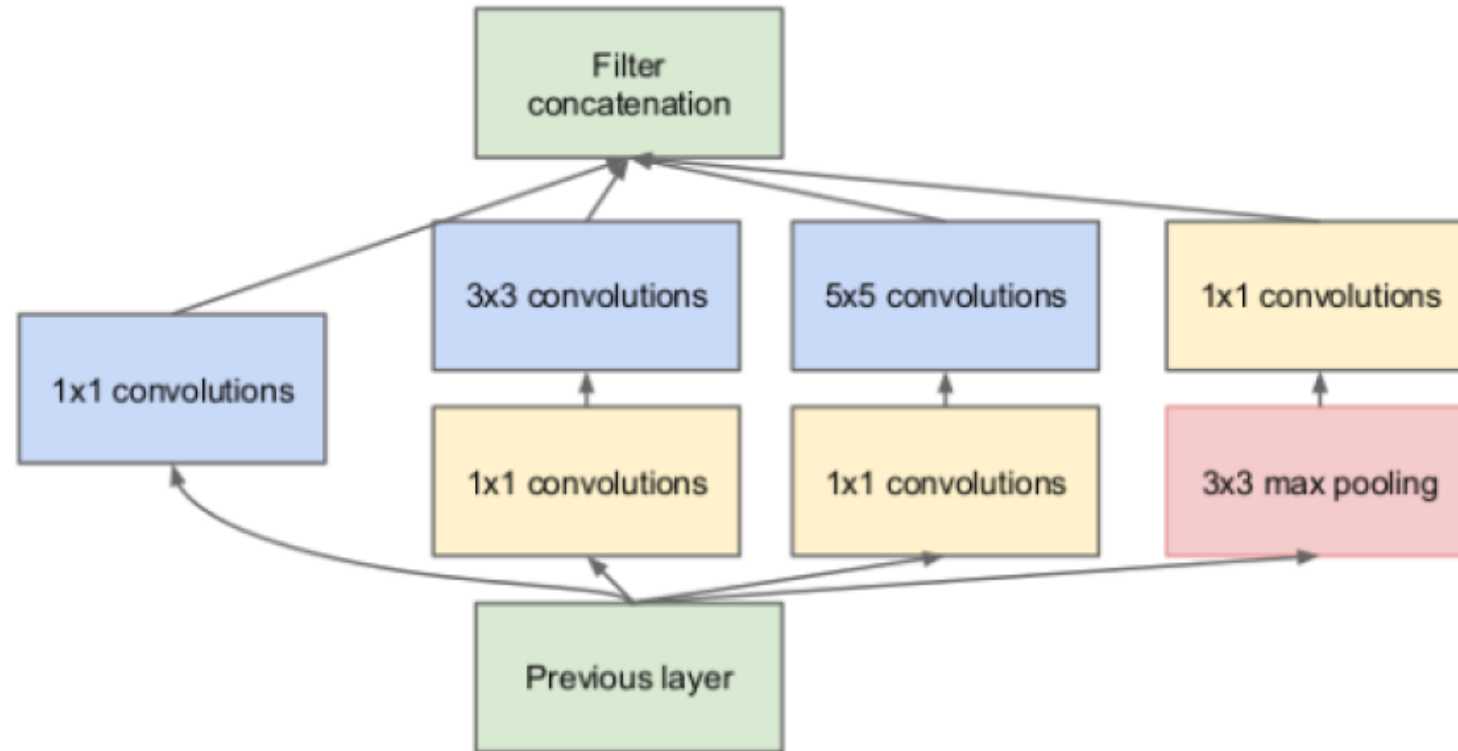


Why: go deeper than VGG without the parameter explosion

Breakthrough: 22 layers yet only ~7M parameters ($\approx 20\times$ fewer than VGG) — ImageNet 2014 winner

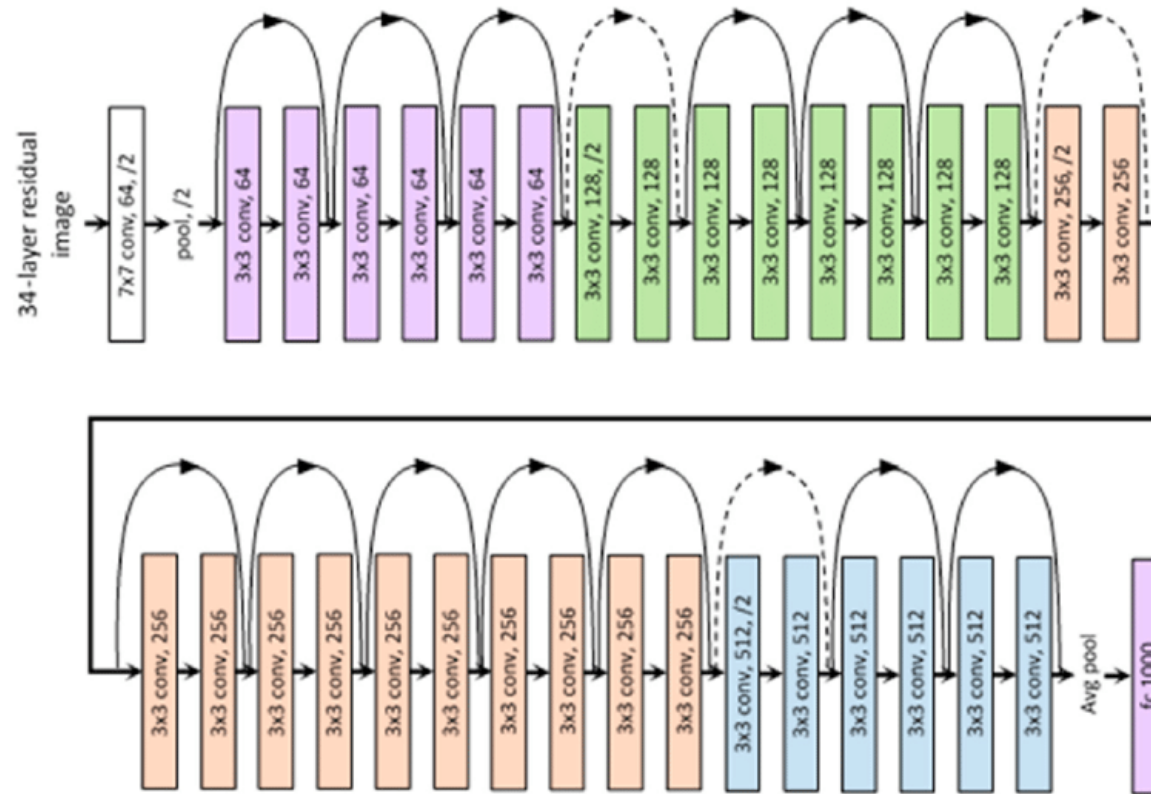
Limitation: complex, hand-crafted design — the trick is the Inception module (next slide)

Inception Module

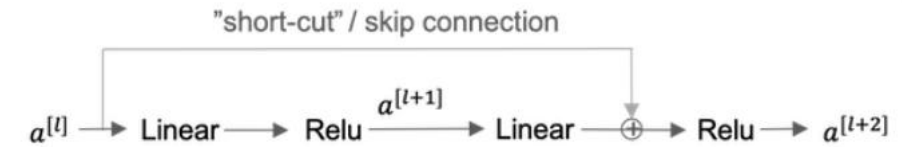


Multi-scale in parallel: 1x1, 3x3, 5x5 convolutions + pooling, outputs concatenated.
The 1x1 convs shrink channels first, so the expensive 3x3 / 5x5 stay cheap.

ResNet Architecture



Residual block



$$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]}$$

$$a^{[l+1]} = g(z^{[l+1]})$$

$$z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}$$

$$a^{[l+2]} = g(z^{[l+2]})$$

$$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]}$$

$$a^{[l+1]} = g(z^{[l+1]})$$

$$z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}$$

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

Why: beyond ~20 layers, deeper plain nets trained worse — “degradation”, and it isn’t overfitting

Breakthrough: skip connections let layers learn residuals; gradients flow through shortcuts → 152 layers, ImageNet 2015 winner (3.6% top-5; human ≈ 5%)

Limitation: compute-heavy at extreme depth — yet still today’s default CNN backbone

Five Architectures, One Story

Model	Year	Layers	Params	Key idea	Limitation
LeNet-5	1998	~5	60K	Conv + pool + FC recipe, trained by backprop	Toy scale — digits only
AlexNet	2012	8	60M	ReLU + dropout + GPUs — deep learning at scale	Ad-hoc design; bulky FC layers
VGG-16	2014	16	138M	Uniform 3×3 stacks → disciplined depth	Params & compute explode
GoogLeNet	2014	22	~7M	Inception: multi-scale + 1×1 bottlenecks	Complex, hand-crafted
ResNet	2015	152	60M	Skip connections fix degradation → 100+ layers	Compute-heavy at extreme depth

One story: deeper + smarter connections — not just bigger. ImageNet top-5 error: ~26% (2011, classical CV) → 15.3% (AlexNet) → 3.6% (ResNet).

Other applications

- Object detection
- Image captioning

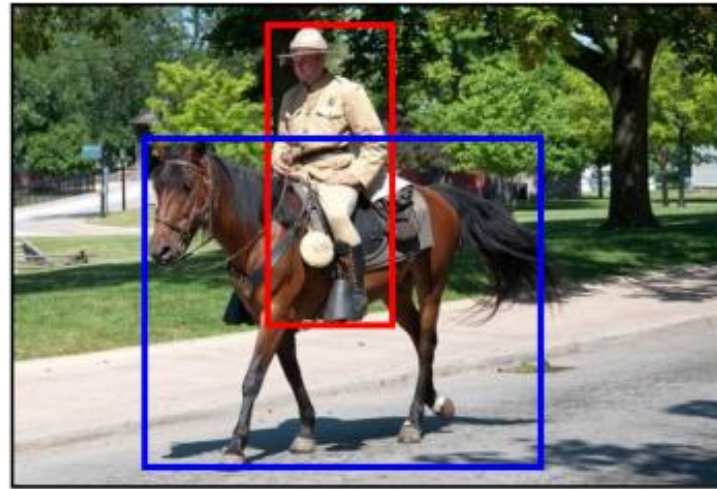
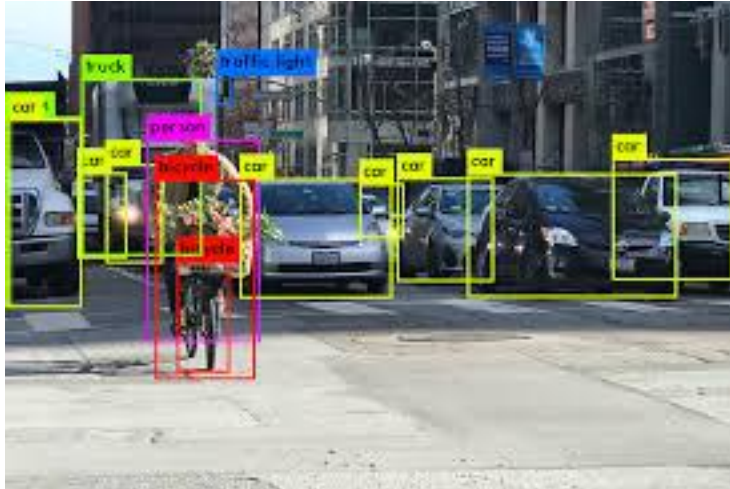


Image Sentence Captioning

A man in uniform riding a brown horse.

Image Paragraph Captioning

A brown horse walking on the road. A man wearing a uniform and a hat. He is riding the horse. There are some trees in the distance.



UNIVERSITY OF
TORONTO

Engineering

Recurrent Networks

Motivation

- ❖ Can we use the previous architectures for sequential inputs?
- ❖ Networks should be able to remember the data over time.
- ❖ Networks should handle varying input length.

Examples

- ❖ Speech Recognition
- ❖ Music Generation
- ❖ Sentiment Classification
- ❖ DNA Sequence analysis
- ❖ Machine translation
- ❖ Named Entity recognition

Notation

❖ Example: Named Entity Recognition

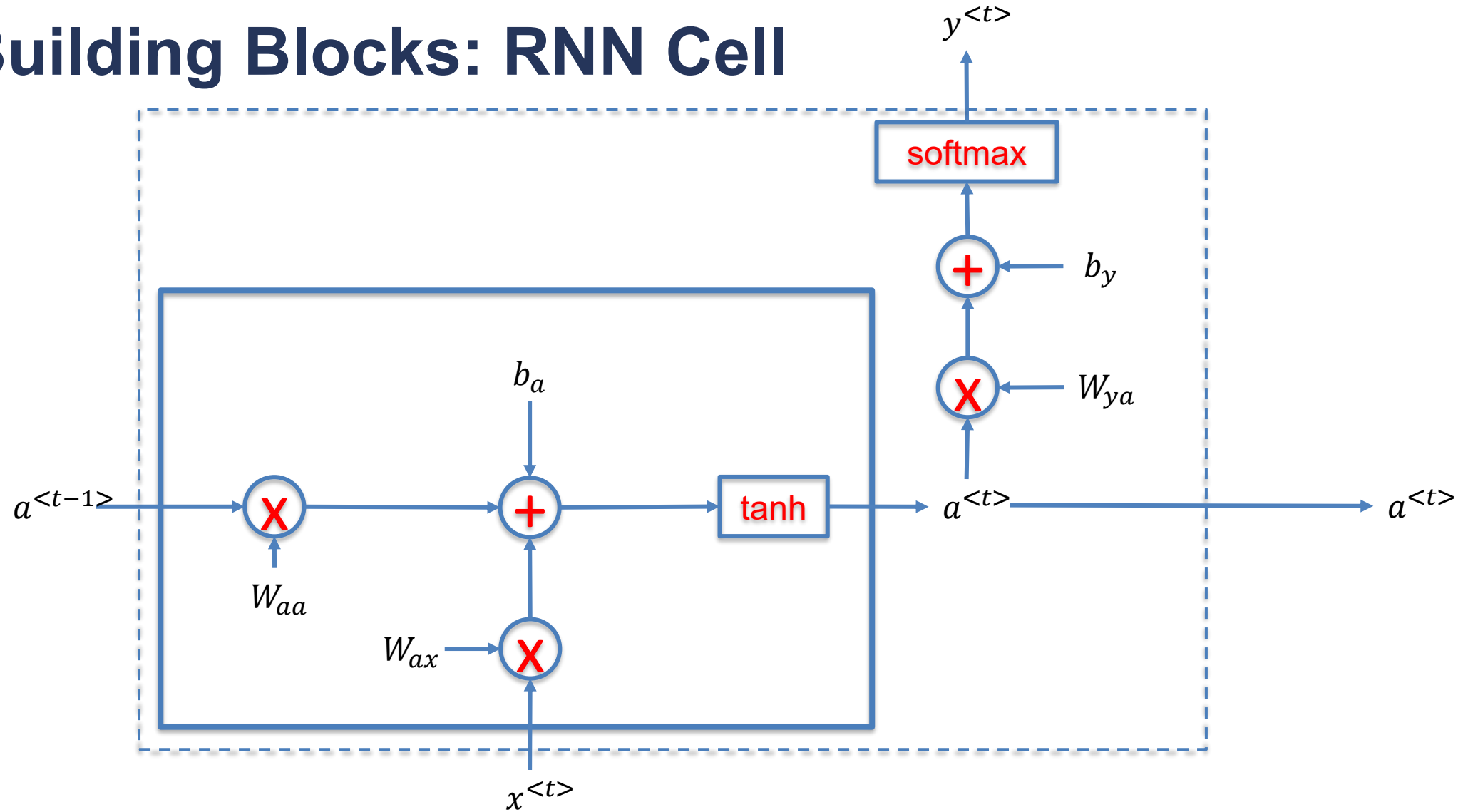
- Input: Harry Potter and Hermione Granger invented a new spell.
- Output: 1 1 0 1 1 0 0 0 0
- $x^{<t>}$ denotes the input in the index t and $y^{<t>}$ denotes the corresponding output.
- In this case, the input and the output length is the same.

Notation

❖ Example: Sentiment Classification

- Input: The current government is not working to benefit its public.
- Output (0 or 1): 0
- $x^{<t>}$ denotes the input in the index t , and y denotes the corresponding output.
- In this case, we only have a single output (i.e., the sentiment).

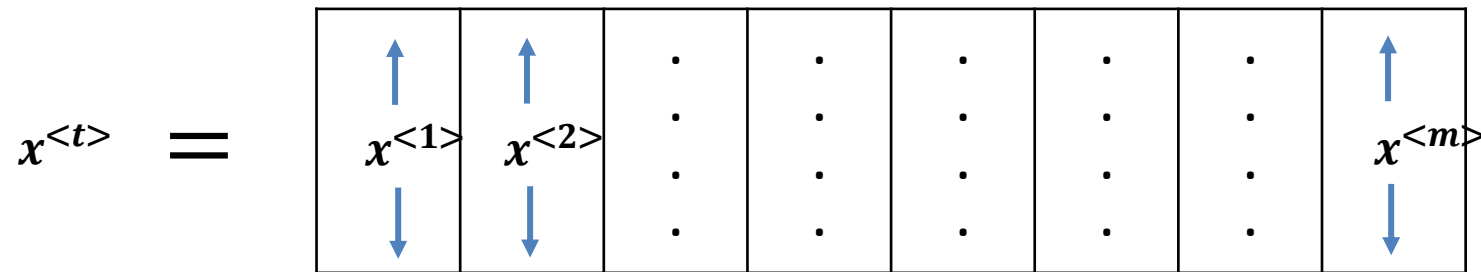
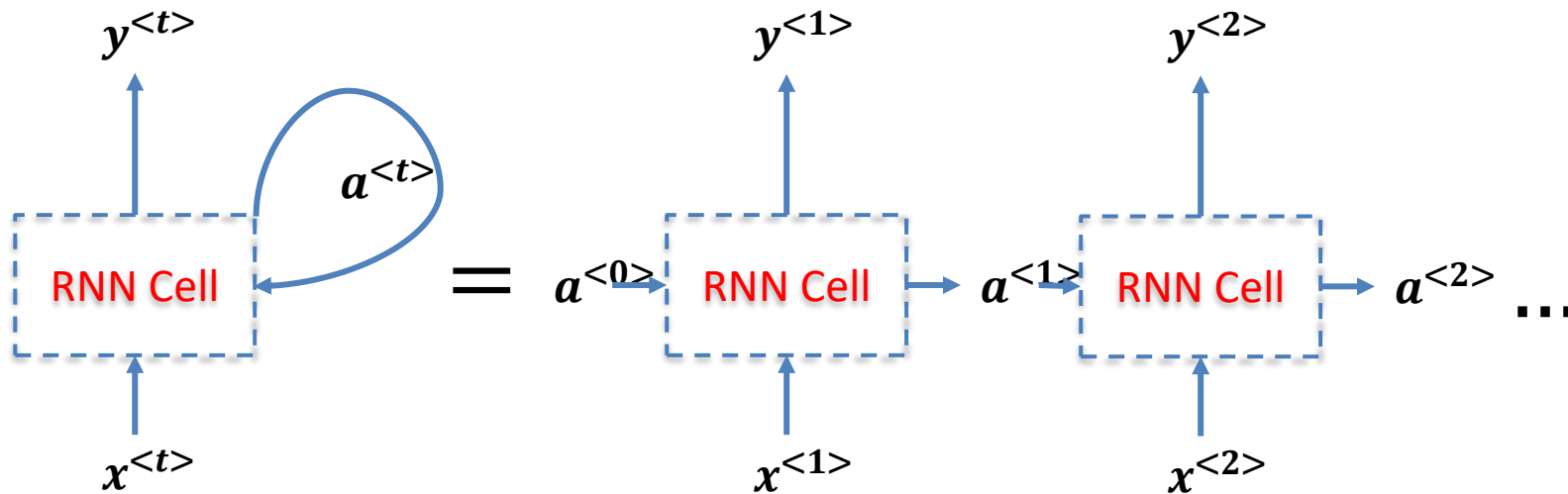
Building Blocks: RNN Cell



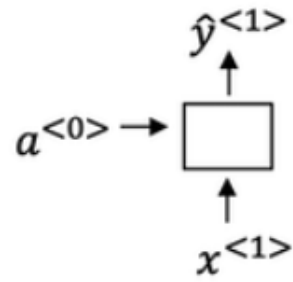
Building Blocks: Unfold and Parameter Sharing

$$a^{<t>} = \tanh(W_{ax}x^{<t>} + W_{aa}a^{<t-1>} + b_a)$$

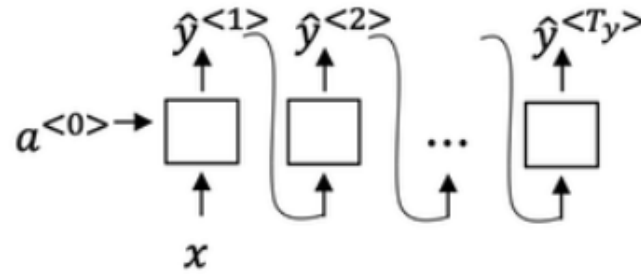
$$y^{<t>} = \text{softmax}(W_{ya}a^{<t>} + b_y)$$



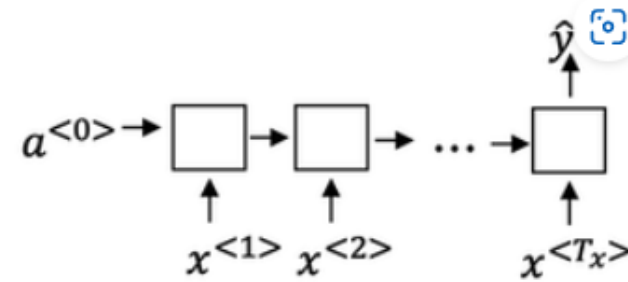
Different types of RNNs



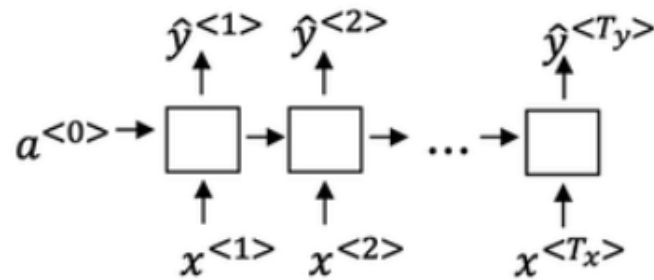
One to one



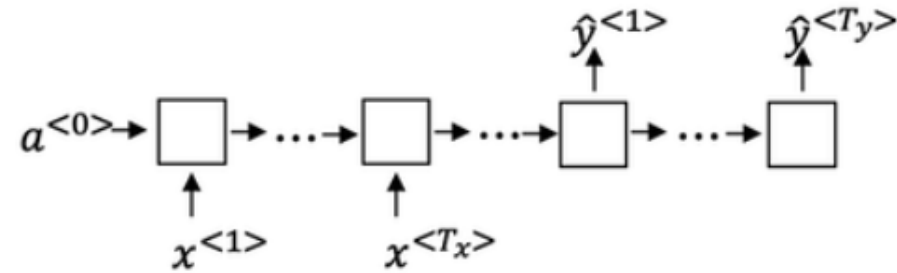
One to many



Many to one



Many to many



Many to many

Challenges with RNNs

- ❖ Vanishing Gradient Problems
- ❖ Not good at capturing very long-term dependencies.
- ❖ It is possible to solve exploding gradient problems using the technique known as gradient clipping. But it is highly challenging to deal with vanishing gradients.

Long Short Term Memory (LSTM)

LSTM

$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c)$$

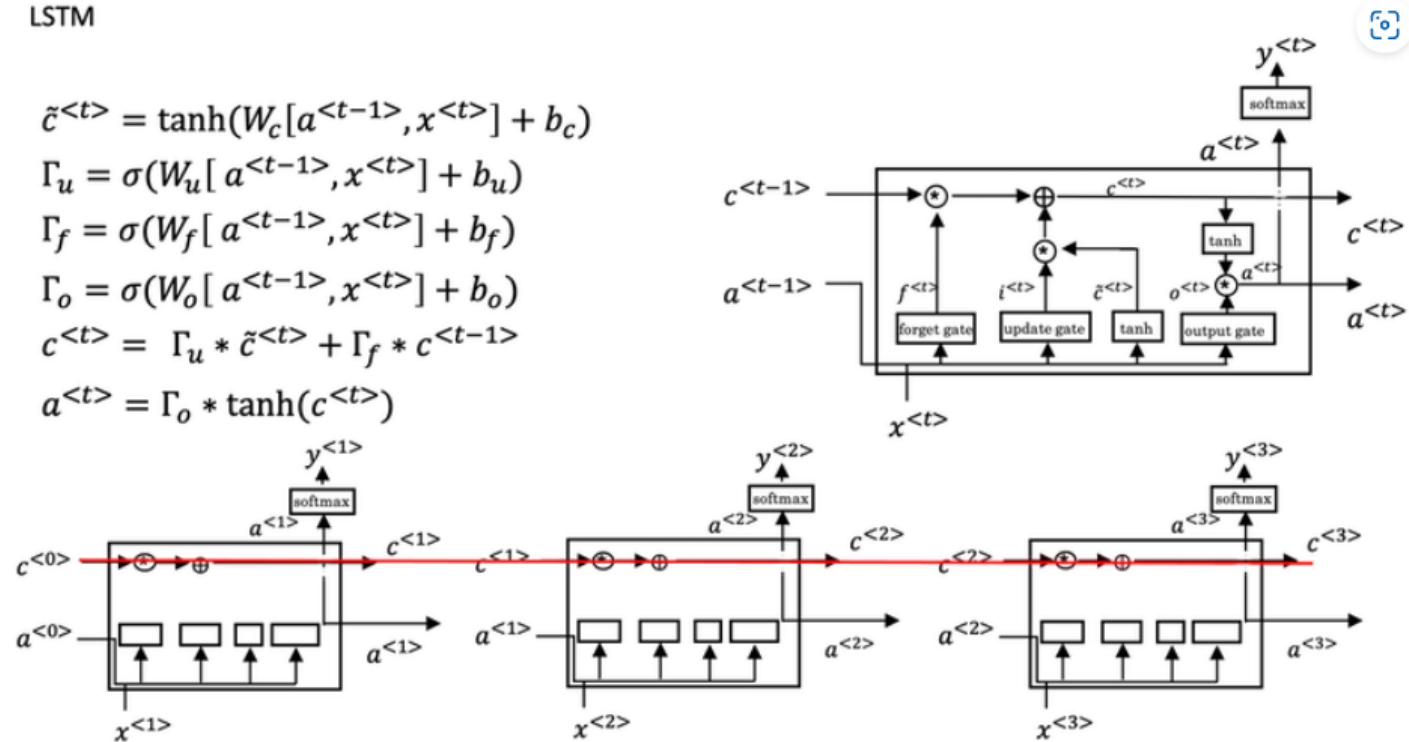
$$\Gamma_u = \sigma(W_u[a^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_f = \sigma(W_f[a^{<t-1>}, x^{<t>}] + b_f)$$

$$\Gamma_o = \sigma(W_o[a^{<t-1>}, x^{<t>}] + b_o)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

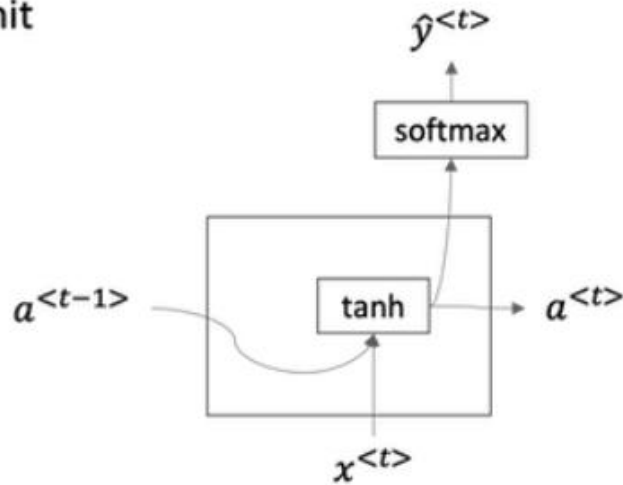
$$a^{<t>} = \Gamma_o * \tanh(c^{<t>})$$



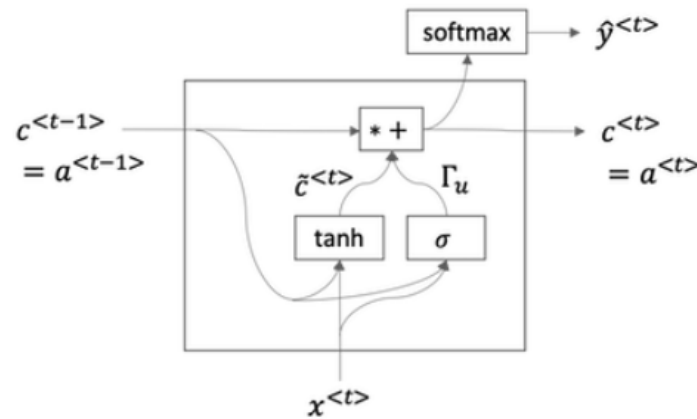
LSTM — Hochreiter & Schmidhuber, 1997. The original gated RNN, designed to fight vanishing gradients with three gates: update, forget, output.

Gated Recurrent Units

RNN unit



GRU



$$\tilde{c}^{<t>} = \tanh(W_c[c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[c^{<t-1>}, x^{<t>}] + b_u)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

Full GRU:

$$\tilde{c}^{<t>} = \tanh(W_c[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r[c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

GRU — Cho et al., 2014. A simplified descendant of the LSTM you just saw: two gates (update, reset) instead of three — fewer parameters, faster.

LSTM vs GRU — Equations Side by Side

LSTM units



GRU

LSTM

$$\tilde{c}^{<t>} = \tanh(W_c[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[c^{<t-1>}, x^{<t>}] + b_u)$$

(update) $\Gamma_u = \sigma(W_u[a^{<t-1>}, x^{<t>}] + b_u)$

$$\Gamma_r = \sigma(W_r[c^{<t-1>}, x^{<t>}] + b_r)$$

(forget) $\Gamma_f = \sigma(W_f[a^{<t-1>}, x^{<t>}] + b_f)$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

(output) $\Gamma_o = \sigma(W_o[a^{<t-1>}, x^{<t>}] + b_o)$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

$$a^{<t>} = \Gamma_o * \tanh(c^{<t>})$$

GRU vs LSTM — Which One?

GRU (Cho et al., 2014)

- 2 gates (update, reset) → fewer parameters
- Trains faster — easier to scale to bigger networks
- Strong default for small & medium datasets
- Often matches LSTM performance in practice

LSTM (Hochreiter & Schmidhuber, 1997)

- 3 gates (update, forget, output) → more expressive
- Separate memory cell c and hidden state a
- Better on long, complex dependencies
- The historically proven default choice

Rule of thumb: no universal winner — try GRU for speed, switch to LSTM if you need more capacity.

Shared limitation: both still read a sequence one step at a time — no parallelism over time, and very long-range context stays hard. Exactly the pain point that attention & Transformers solve → Day 8.



UNIVERSITY OF
TORONTO

Engineering

Thank you.